Block vs File Storage

a disk for one machine, a folder for many

Watch the 4-min video

In one line: block storage is a raw, low-latency disk attached to one machine (databases, brokers); file storage is a shared file system that many machines mount over NFS or SMB; object storage is whole objects over HTTP. Size block volumes by **IOPS** for random I/O and file or object storage by **throughput** for big sequential reads.

1 Two problems object storage won't solve

A Postgres database runs on cheap network disks and every query crawls: `iostat` shows the disk at 100% busy with single-digit-MB/s reads. Meanwhile ten GPU servers each download the same 40 GB model from a bucket every time they start, so scaling out takes minutes. A database needs a **disk** it can read and write 8 KB at a time at random offsets with `fsync`; an object store only replaces whole objects. The model servers need a **shared folder**, not 10 private copies. Different access patterns, different storage.

2 The three models

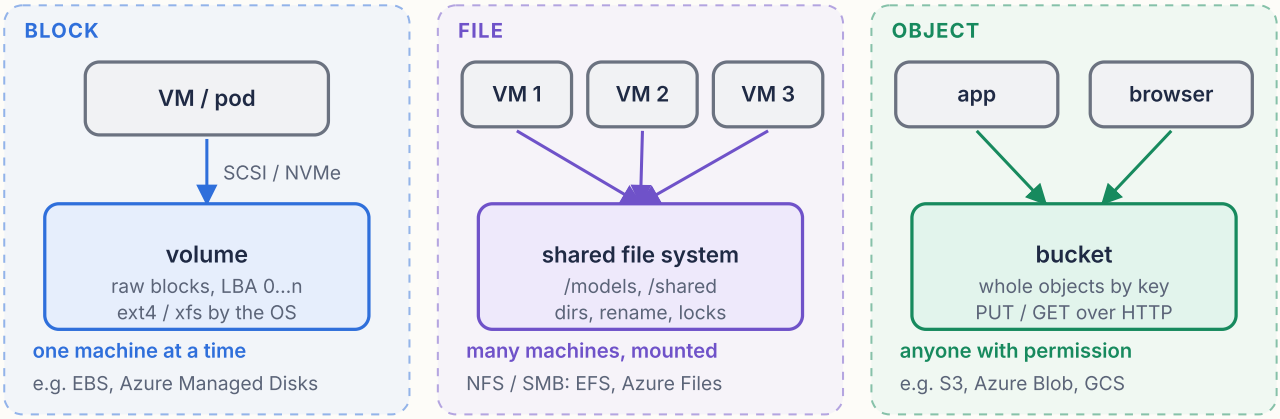


Figure 1. Block storage is a device; file storage is a network file system; object storage is an HTTP service. The interface decides what software can use it.

Block storage: a raw disk

The volume is an array of fixed-size blocks addressed by number. The operating system puts a file system on it (ext4, xfs, NTFS) and owns everything above the blocks: directories, caching, locking. Because only one OS manages that file system, a volume is normally attached to **one machine at a time**. Cloud examples: **Amazon EBS**, **Azure Managed Disks**, Google Persistent Disk; in Kubernetes a `ReadWriteOnce` PersistentVolume backed by a CSI driver.

- **Network-attached** (EBS, Managed Disks): survives instance failure, can be detached and re-attached, but lives in one Availability Zone (Azure also offers zone-redundant SSD disks). Snapshots are incremental and stored durably (EBS snapshots live in S3).
- **Local NVMe / instance store**: far lower latency and higher IOPS, but **ephemeral**: the data is gone when the instance stops or fails. Fine for caches, scratch space or replicated systems (Cassandra,

Kafka with replication) that can rebuild a node.

- **Multi-attach** exists (EBS io1/io2 to up to 16 instances in one AZ, Azure shared disks) but only with a cluster-aware file system or application; two ordinary ext4 mounts of one disk will corrupt it.

File storage: a shared file system

A server (or managed service) exposes directories over **NFS** (v3, v4.1; Linux) or **SMB** (3.x; Windows, also Linux). Many clients mount the same tree and get ordinary file semantics: open, seek, append, rename, permissions and locks. Managed versions: **Amazon EFS** (NFSv4.1, elastic capacity, multi-AZ), **Azure Files** (SMB, plus NFS on premium shares), **Google Filestore**; for high-throughput ML and HPC, **FSx for Lustre** or Azure Managed Lustre. In Kubernetes: `ReadWriteMany` volumes.

The cost is per-operation latency: every `open`, `stat` or directory listing is a network round trip, so workloads with millions of tiny files (a `node_modules` tree, a `git checkout`) are much slower than on a local disk.

Tools like **Mountpoint for Amazon S3** or **BlobFuse** mount a bucket as a folder. Good for reading large files, but not full POSIX: no in-place edits, limited renames, no locks.

❸ IOPS, throughput and latency

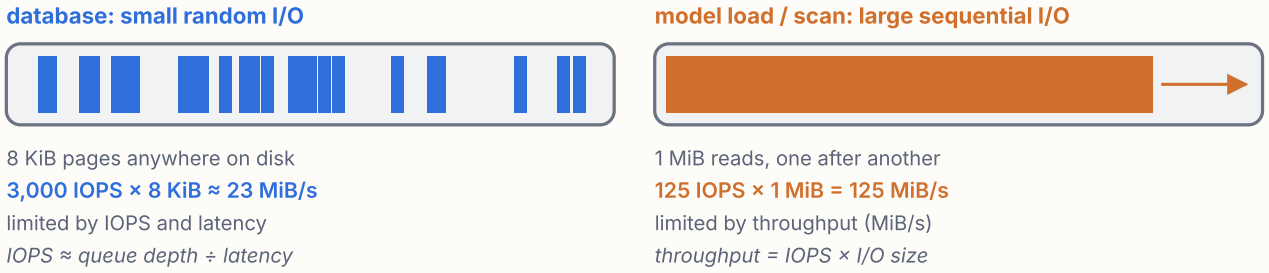


Figure 2. The same disk can be the bottleneck in two different ways. Work out which number your workload needs before choosing a tier.

- **IOPS**: I/O operations per second. Databases read and write 8 KiB (Postgres) or 16 KiB (InnoDB) pages at random places, so they are IOPS-bound.
- **Throughput**: MB per second. Loading model weights, scanning Parquet files or streaming video uses large sequential reads, so it's throughput-bound. **Throughput = IOPS × I/O size.**
- **Latency**: time per operation. With one request in flight, 1 ms latency caps you at 1,000 IOPS; by Little's law, IOPS ≈ outstanding requests (queue depth) ÷ latency. That's why a single-threaded workload can't use the IOPS you paid for, and why commit latency (`fsync` of the write-ahead log) matters more than peak IOPS for a busy OLTP database.

Volume type (examples)	Medium	Performance model	Good for
EBS gp3	SSD	3,000 IOPS + 125 MiB/s baseline at any size; buy more of each independently	most databases and boot volumes
EBS io2 Block Express	SSD	provisioned IOPS, up to 256,000; sub-millisecond latency	large, latency-critical databases
EBS st1 / sc1	HDD	throughput credits; poor at random I/O	big sequential data: logs, cold data
Azure Premium SSD (v1)	SSD	performance tied to size (e.g. P30, 1 TiB: 5,000 IOPS, 200 MB/s)	databases; buy size for IOPS
Azure Premium SSD v2 / Ultra Disk	SSD	IOPS and throughput set independently of size	databases needing tuning
Local NVMe (instance store)	SSD	hundreds of thousands of IOPS, ~100 µs; ephemeral	caches, replicated stores, scratch

Two more limits catch people out: the **instance** has its own maximum storage bandwidth and IOPS (a small VM can't drive a big io2 volume), and network file systems often scale throughput with stored size or a provisioned/elastic mode, so a small share may be slow.

4 Side by side

	Block	File	Object
Interface	device (SCSI/NVMe); OS file system on top	NFS / SMB mount, POSIX-like	HTTP API: PUT, GET, LIST
Sharing	one machine (normally)	many machines at once	any number of clients
Updates	any block, in place	in place, append, rename, locks	replace whole object
Latency	lowest: sub-ms to a few ms	ms per operation (network round trip)	tens of ms to first byte
Scale	volume size limit (EBS: 16–64 TiB)	elastic up to petabytes	practically unlimited
≈ price per GB-month*	$0.08 (gp3)	$0.30 (EFS Standard)	$0.023 (S3 Standard)
Typical use	databases, brokers, boot disks	shared content, home dirs, ML datasets, legacy apps	media, backups, data lakes, artifacts

*Approximate us-east-1 list prices; extra IOPS/throughput and requests are billed on top.

5 How to choose

1. **Does the software expect a local disk** (a database, Kafka, Elasticsearch, a VM boot volume)? Use **block**. Size it by IOPS and latency, not just capacity.
2. **Do several machines need the same files through normal file operations** (shared uploads in a legacy app, home directories, a model or dataset read by many pods)? Use **file**.
3. **Everything else**, especially large amounts of data written once and read by key (media, backups, logs, data lakes): use **object**. It's the cheapest and the easiest to scale and share.

Real systems use all three at once. A common pattern: object storage is the durable source of truth, and block or file storage is a fast working copy (a database's volume backed up to a bucket; a file share or local NVMe cache filled from a bucket).

6 Example: Postgres + shared model weights

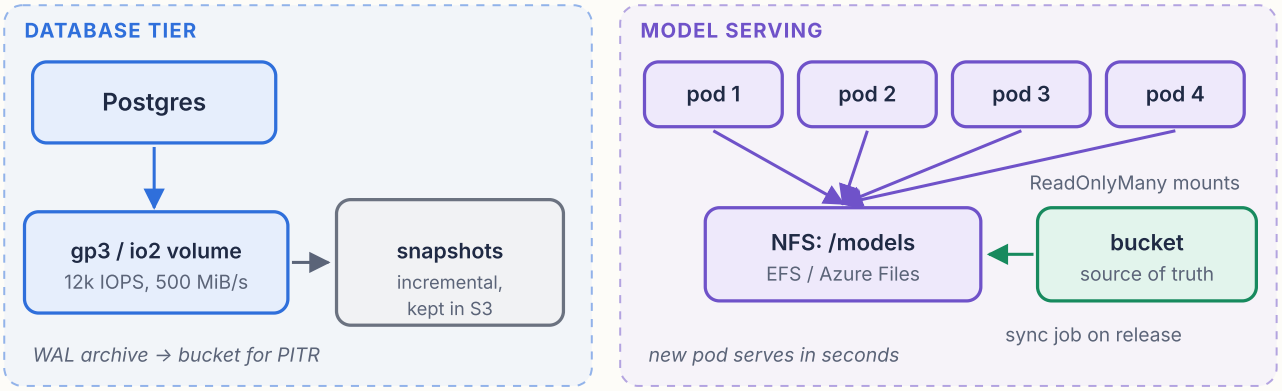


Figure 3. Each workload gets the storage its access pattern needs; object storage stays the durable source for both.

WORKED EXAMPLE: ONE PLATFORM, THREE KINDS OF STORAGE

Postgres (500 GB, peaks of ~10,000 random IOPS) runs on an SSD block volume: gp3 provisioned at 12,000 IOPS and 500 MiB/s, or io2 if commit latency must stay sub-millisecond. Volume snapshots give quick restores, and continuous WAL archiving to a bucket gives point-in-time recovery (snapshots alone are only crash-consistent).

Model serving: the 40 GB weights live in a bucket. On each release a job copies them to an NFS share (EFS / Azure Files), and every pod mounts `/models` read-only. Before: 10 pods × 40 GB = 400 GB downloaded per rollout; at 125 MiB/s one download takes 40 × 1024 ÷ 125 ≈ 330 s, about 5½ minutes per pod. After: a new pod mounts the share and starts loading at once, and the operating system's page cache keeps hot files in memory. Check the share's throughput mode: ten pods loading at once share its bandwidth.

TRADE-OFFS

- Block: fastest and simplest for one machine, but tied to one zone and one host; HA needs replication at the application level.
- File: easy sharing with familiar semantics, but per-operation latency, higher price per GB and lock/consistency subtleties (NFS close-to-open consistency).
- Object: cheapest and most durable, but no in-place edits and higher latency; apps must use its API or a limited mount.
- Local NVMe: unbeatable speed, zero durability; only for data you can rebuild.

COMMON MISTAKES

- Running a database on NFS or a bucket mount: slow commits, and fragile locking and `fsync` semantics.
- Sizing a volume by gigabytes and ignoring IOPS, latency and the instance's own storage bandwidth limit.
- Keeping irreplaceable data on ephemeral instance storage.
- Treating a crash-consistent volume snapshot as a complete database backup strategy (no point-in-time recovery, no restore test).

7 Key terms

Block storage: raw volume of numbered blocks, formatted by one OS

File storage: shared file system mounted over NFS or SMB

IOPS: I/O operations per second; what random-access workloads need

Throughput: MB/s; what large sequential reads need

Queue depth: outstanding I/Os; IOPS ≈ queue depth ÷ latency

ReadWriteOnce / ReadWriteMany: Kubernetes access modes for block vs shared volumes

8 Interview questions

Q1. When do you need block storage vs object storage? BASIC

Block storage when software needs a low-latency disk with random reads and writes, like a database or broker; object storage for large amounts of files accessed by key, cheaply and durably.

Q2. What's the difference between IOPS and throughput? BASIC

IOPS is operations per second (small random access, what databases need); throughput is MB per second (large sequential reads). Throughput = IOPS × I/O size.

Q3. Why can't two servers mount the same EBS volume and share files? BASIC

Each OS caches and manages the file system as if it owned the disk, so two writers corrupt it; use a shared file system (NFS/SMB) or a cluster-aware file system with multi-attach.

Q4. Your Postgres p99 latency spikes and the volume shows 3,000 IOPS flat. What's happening and what do you do? SCENARIO

It's hitting the gp3 baseline of 3,000 IOPS. Provision more IOPS (or move to io2), check the instance's EBS limit, and reduce I/O with more RAM for cache or better indexes.

Q5. Ten inference pods each download a 40 GB model at start-up. How would you speed this up? SCENARIO

Keep the model in a bucket as the source, but serve it from a shared read-only file system or a node-local cache, so new pods read it instead of each downloading 40 GB.

Q6. You provisioned 64,000 IOPS but a single-threaded job gets 1,500. Why? TRICKY

With one I/O in flight at ~0.6 ms latency, it can do at most ~1,600 IOPS (Little's law); increase concurrency (queue depth) or batch I/O.

Q7. Can you run a database on S3 via a FUSE mount? TRICKY

Not safely: bucket mounts lack in-place writes, locks and real fsync semantics, at tens of ms per operation. Databases that tier to object storage do it inside their own engine.

Next: SD-O-003 · Blob Storage Patterns backendroadmap.com